Microsoft Agent Framework & Microsoft Foundry Agent Service

Nov 10, 2026 · @V

1. Overview

Microsoft Agent Framework is the open-source SDK you write agents with; Microsoft Foundry Agent Service is the managed Azure platform that runs, governs and scales them. The two are designed as a pair: you build in Agent Framework on a laptop, then deploy the same code to Foundry as a hosted agent.

Microsoft Agent Framework is an open-source development kit for building AI agents and multi-agent workflows in .NET, Python and (in preview) Go. It is the direct successor to Semantic Kernel and AutoGen, built by the same teams. It combines AutoGen's simple agent abstractions with Semantic Kernel's enterprise features (session state, type safety, middleware, telemetry) and adds graph-based workflows for explicit multi-agent orchestration (overview). Both predecessors are now in maintenance mode.

Microsoft Foundry Agent Service is a managed platform for building, deploying and scaling AI agents with any framework and any supported model from the Foundry catalog. It offers three ways in: declarative prompt agents that Foundry runs for you, code-based hosted agents packaged as containers, and the Responses API for agents you already run elsewhere (overview). Foundry was formerly branded Azure AI Foundry, and the agent service was earlier called Azure AI Agent Service.

	Microsoft Agent Framework	Microsoft Foundry Agent Service
What it is	Open-source SDK and runtime library (MIT)	Managed Azure PaaS
Where it runs	Anywhere: laptop, container, any cloud	Azure, inside a Foundry project
Languages	.NET, Python (GA); Go (preview)	Portal, REST, SDKs; hosted code in any framework
Main abstractions	Agents, sessions, tools, middleware, context providers, workflows, harness	Prompt agents, hosted agents, toolboxes, conversations, runtime

	Microsoft Agent Framework	Microsoft Foundry Agent Service
Models	Foundry, Azure OpenAI, OpenAI, Anthropic, Bedrock, Gemini, Ollama and more	Foundry model catalog
You pay for	Your own model calls and compute	Inference + tool usage (+ container compute for hosted agents)

Release history

Date	Milestone
Aug 2026	Agent Harness and Foundry Hosted Agents reach general availability (InfoQ)
Jun 2026 (Build)	Agent Harness, GitHub Copilot SDK and Claude Agent SDK connectors, and multi-agent orchestration patterns (incl. Magentic) go stable
Apr 3, 2026	Agent Framework 1.0 GA for .NET and Python, with stable APIs and a long-term support commitment (Visual Studio Magazine)
Feb 2026	Release candidate: API surface declared stable and feature-complete for 1.0 (InfoQ)
Oct 2025	Agent Framework introduced in public preview as the unification of Semantic Kernel and AutoGen

Note: some Microsoft Learn pages still label hosted agents as preview; check the regional availability page before planning production use.

2. Agent Framework: core concepts

An Agent Framework agent combines an agent abstraction, a model or remote-agent connection, instructions, tools, middleware, context providers and session state behind one run interface (agent concepts). Everything else in the framework composes these pieces.

Concept	.NET type	Python type	What it does
Agent	`AIAgent`, `ChatClientAgent`	`Agent`	The unit you call with `RunAsync()` / `run()`; streaming and non-streaming
Chat client	`IChatClient` (Microsoft.Extensions.AI)	`FoundryChatClient`, `OpenAIChatClient`, `AnthropicClient` …	Provider-specific connection to an LLM
Remote / direct agents	`A2AAgent`, `GitHubCopilotAgent`, `CopilotStudioAgent`	`FoundryAgent`, `A2AAgent`, `GitHubCopilotAgent`, `ClaudeAgent`	Proxies for agents that run in another runtime
Session	`AgentSession`	session object	Holds multi-turn conversation state (replaces SK/AutoGen "threads")
Tools	`AIFunction`, hosted tools, MCP tools	Python functions, hosted tools, MCP tools	Actions the model can call
Context providers	`ChatHistoryProvider` + `AIContextProviders`	unified `context_providers` list	Load history, inject memory, RAG results or dynamic instructions before each model call
Middleware	`.AsBuilder().Use(...)`	`middleware=[...]`	Intercepts agent runs, model calls and function calls

Concept	.NET type	Python type	What it does
Harness agent	harness builder	`create_harness_agent()`	Batteries-included runtime for long, multi-step tasks
Workflow	`WorkflowBuilder`	`WorkflowBuilder`	Graph of executors and edges for explicit orchestration

Agents and agent types

Chat-client agents are application-owned agents backed by a model inference client. They support function tools, multi-turn conversations, provider-hosted tools, structured outputs, streaming, middleware, and local or service-managed history. Any client implementing `IChatClient` can back a `ChatClientAgent`. All agent types share the `AIAgent` abstraction, so orchestration code can mix chat-client agents, custom agents and remote proxies through one interface.

First-party model connectors cover Microsoft Foundry, Azure OpenAI, OpenAI, Anthropic Claude, Amazon Bedrock, Google Gemini and Ollama (1.0 announcement coverage). Runtime model routing lets a conversation switch models or providers without losing context.

Sessions and conversation state

A session carries the conversation between runs. History can live locally (an in-memory or custom `ChatHistoryProvider`) or in the service, as with Foundry conversations. Sessions can be serialized and restored, which is what makes long-running and human-in-the-loop scenarios possible. Compaction trims or summarizes history when it nears the model's context window.

Tools

- **Function tools**: plain C# methods or Python functions, described to the model from their signatures and docstrings.

- **Provider-hosted tools**: code interpreter, file search, web search, where the provider supports them.
- **MCP tools**: local (stdio) or remote (HTTP) Model Context Protocol servers, discovered dynamically.
- **Agents as tools**: any agent can be wrapped as a function tool for another agent.
- **Tool approval**: tools can require human approval before running; the harness adds a "don't ask again" option.

Context providers and memory

Context providers run before each LLM call. In .NET there is one `ChatHistoryProvider` and a list of `AIContextProviders`; each provider's output feeds the next. In Python a single `context_providers` list holds both history and context providers, and providers can attach per-run chat or function middleware (agent pipeline). The pluggable memory architecture supports chat history, persistent key-value state and vector retrieval, with adapters for Foundry, Mem0, Redis and Neo4j (coverage of the 1.0 release).

Middleware

Middleware works at three levels: agent middleware wraps the whole run (works with any `AIAgent`, including remote ones); chat middleware wraps each model call; function middleware wraps each tool call. Typical uses are logging, PII redaction, input validation, policy checks, caching and overriding results. Agent Hooks install one middleware bundle across all three layers.

Harness agent

The harness is the production runtime that wraps a model so it can call tools and keep working until a job is done. On by default: function invocation, per-call history persistence, context compaction, a todo list with plan and execute modes, file memory, skills, web search, tool approval and OpenTelemetry. Opt-in, with a warning: shell tooling, file access, background sub-agents and automatic looping (InfoQ).

Agents vs workflows

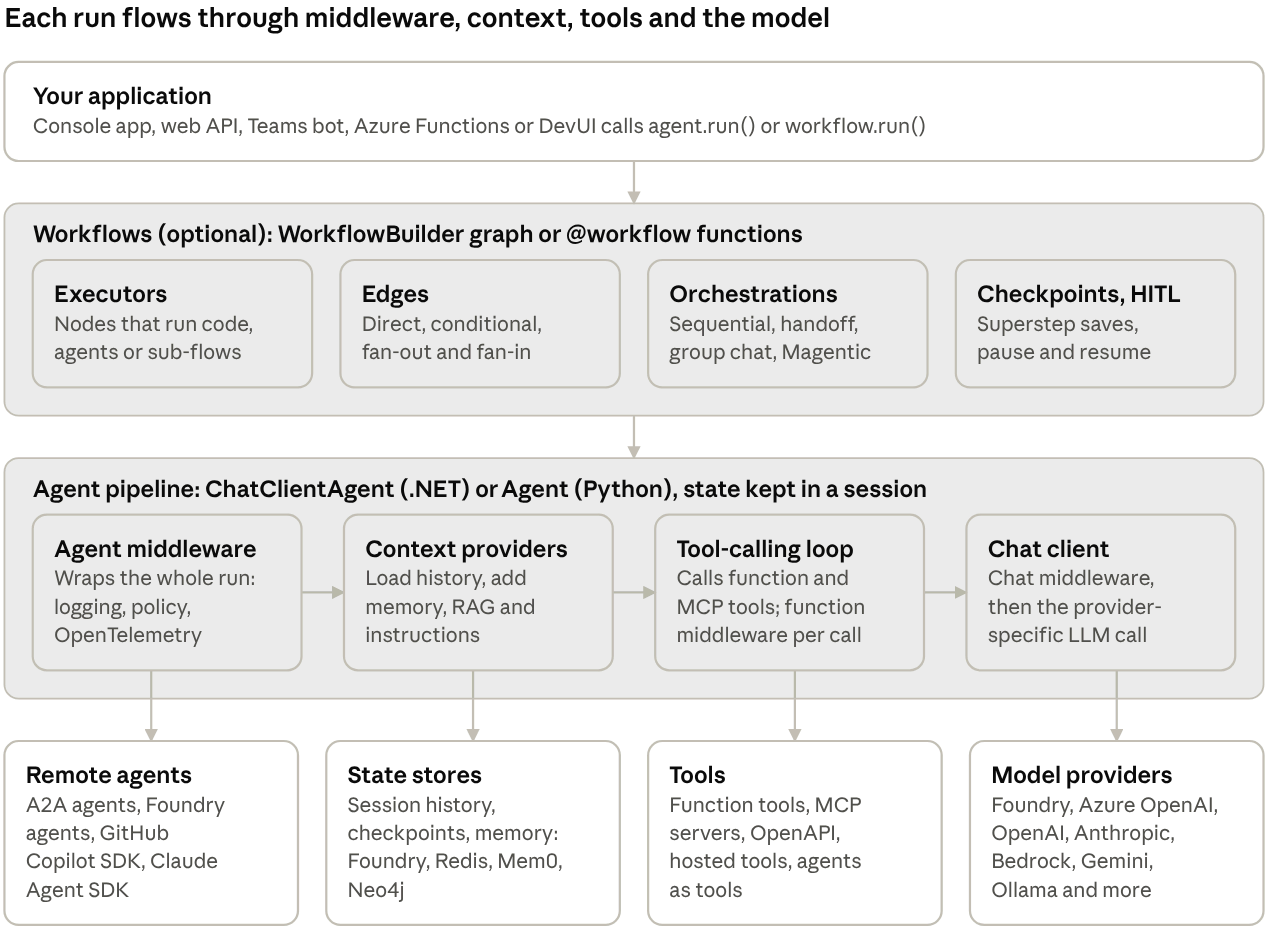
Use an agent when…	Use a workflow when…
The task is open-ended or conversational	The process has well-defined steps

Use an agent when…	Use a workflow when…
You need autonomous tool use and planning	You need explicit control over execution order
One LLM call, possibly with tools, is enough	Multiple agents or functions must coordinate

Microsoft's own rule of thumb: if you can write a plain function for the task, do that instead of using an agent.

3. Agent Framework architecture

Agent Framework is layered: your app calls an optional workflow, the workflow calls agents, and each agent runs a fixed pipeline of middleware, context providers, a tool-calling loop and a chat client before reaching a model, tool, store or remote agent.



Agent Framework architecture · 4 layers

Arrows show the direction of a request; responses flow back up through the same layers, and context providers are notified of new messages so they can persist history and memory (agent pipeline).

What happens during one `run()`

1. **Agent middleware and telemetry** execute (if configured) and open an OpenTelemetry span.
2. **Context providers** load session history, add memories or retrieved documents, and may attach per-run chat or function middleware.
3. The request passes to the **chat client**, whose **function-invocation layer** manages the tool-calling loop; **function middleware** runs around every tool call.
4. **Chat middleware** runs around every individual model call.
5. The **raw chat client** sends the provider-specific request to the LLM.
6. The response flows back through the same layers.
7. **Context providers** are notified of the new messages and persist them.

Layer reference

Layer	.NET	Python	Typical use
Agent middleware	`.AsBuilder().Use(runFunc, runStreamingFunc)`	`middleware=[...]`, `AgentMiddlewareLayer`, `AgentTelemetryLayer`	Audit, guardrails, rate limits; works for remote agents too
History	`ChatHistoryProvider` (one)	history provider in `context_providers`	Local or service-side conversation storage
Context	`AIContextProviders` (list)	context providers in `context_providers`	Memory, RAG, dynamic instructions
Function invocation	Default wrapper on `IChatClient`	`FunctionInvocation` layer	Tool loop, tool approval, per-call function middleware
Chat middleware	`IChatClient.AsBuilder().Use(...)`	chat middleware + telemetry	Caching, prompt rewriting, token accounting

Layer	.NET	Python	Typical use
Raw client	`IChatClient` implementation	`RawChatClient`	Provider-specific wire calls

Remote agent types such as `A2AAgent`, `GitHubCopilotAgent` and `CopilotStudioAgent` skip the local chat-client layers but still accept agent-level middleware, so governance code is written once for every agent type.

4. Workflows and multi-agent orchestration

Workflows define explicit, inspectable execution paths that coordinate code, agents, state, events and human input (workflow concepts). Where an agent decides its own next step, a workflow's graph decides it, which makes runs repeatable, testable and resumable.

Graph primitives

- **Executors**: nodes that receive typed input, do work (run code, call an agent, call a sub-workflow) and emit output. An `AgentExecutor` puts an agent into the graph.
- **Edges**: route messages between executors. Edges can be direct, conditional (switch-case), fan-out (one to many) or fan-in (many to one). Routing is type-validated when the graph is built.
- **Events**: the workflow emits lifecycle, executor and agent-update events that you stream to a UI or log.
- **State**: run-scoped and durable shared state that executors read and write.
- **Supersteps**: executors that are ready run in parallel in a superstep; checkpoints are taken at superstep boundaries.

Two APIs over one run model

	Functional (`@workflow`, Python, experimental)	Graph (`WorkflowBuilder`, .NET / Python / Go)
Control flow	Native Python (`if`, loops, `asyncio.gather`)	Edges and conditions
Best for	Sequential pipelines, custom loops, ad-hoc parallelism	Fixed graphs, fan-out/fan-in, type-validated routing

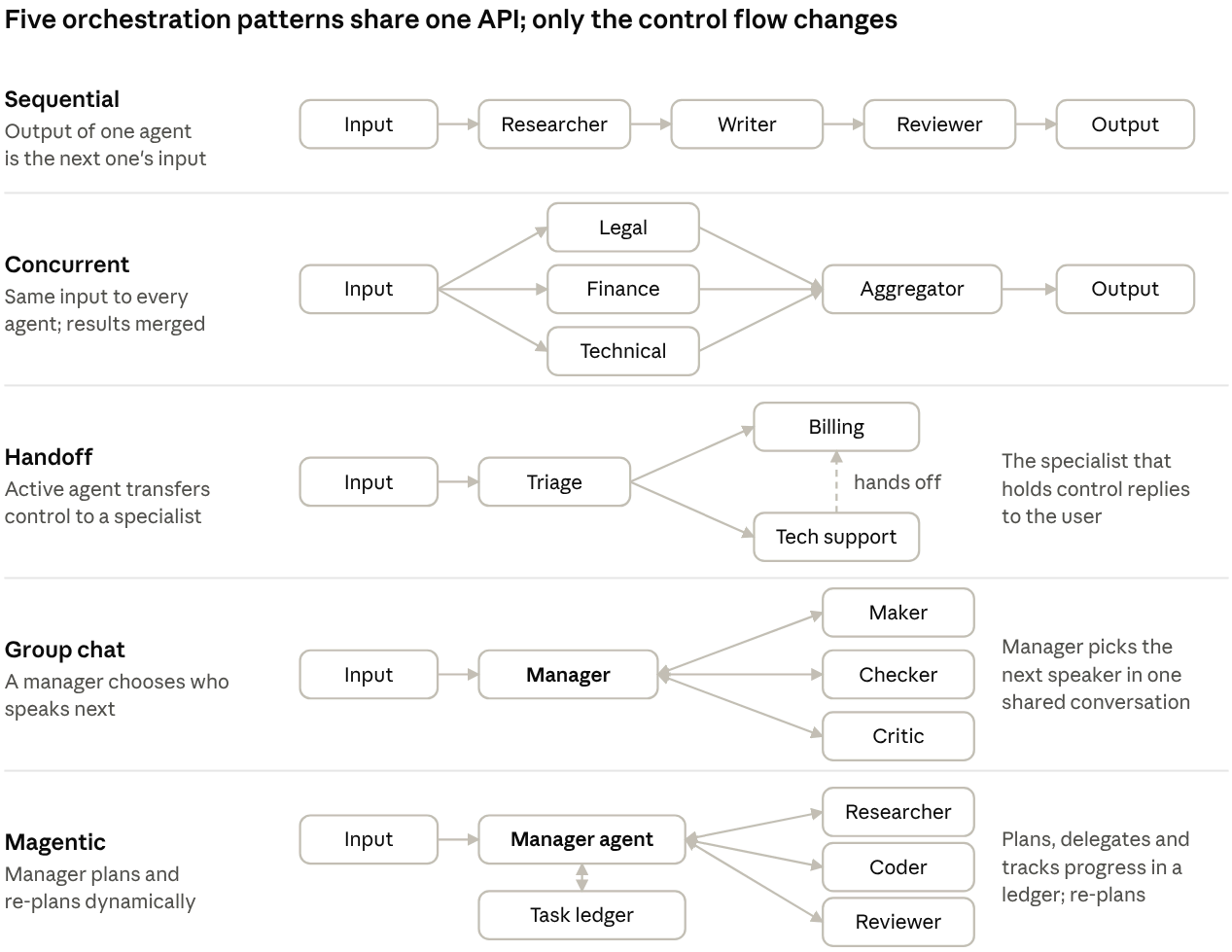
	Functional (`@workflow` , Python, experimental)	Graph (`WorkflowBuilder` , .NET / Python / Go)
Human-in-the-loop	`ctx.request_info()`	`RequestInfoExecutor`
Checkpointing	Per- `@step` result caching	Superstep-boundary checkpoints
Expose as an agent	`.as_agent()`	`.as_agent()` / `AsAIAgent()`

Because a workflow can be wrapped as an agent, a whole multi-agent graph can be used anywhere a single agent can: as a tool, in another workflow, or deployed to Foundry as a hosted agent.

Built-in orchestration patterns

All patterns share one API, so you can change coordination style without rewriting the agents. All support streaming, checkpointing, human-in-the-loop approvals and pause/resume (1.0 coverage).

Pattern	How control moves	Good for
Sequential	Each agent's output is the next agent's input, in a fixed order	Draft → review → polish pipelines, document processing
Concurrent	The same input fans out to all agents in parallel; results are aggregated	Multiple expert opinions, ensemble answers, parallel research
Handoff	The active agent transfers control to a specialist based on context	Customer-service triage, routing to billing / tech / returns
Group chat	A manager selects which agent speaks next in a shared conversation	Brainstorming, debate, maker-checker review loops
Magentic	A manager agent plans, keeps a task ledger, delegates to workers and re-plans	Open-ended, complex tasks where the steps are not known up front



Orchestration patterns · 5 control flows

Agent names are examples; any agents, including remote A2A agents or whole sub-workflows, can be participants. Two-headed arrows mark back-and-forth turns with the manager.

The Magentic pattern derives from Microsoft Research's Magentic-One, whose 2024 evaluation reported 38% on GAIA, 27.7% on AssistantBench and 32.8% on WebArena (InfoQ).

Checkpointing and human-in-the-loop

A checkpoint store saves executor state, pending messages and shared state at each superstep boundary. A run can be paused, persisted and resumed later, even in another process, which is how a workflow waits hours or days for a human approval. A `RequestInfoExecutor` (graph API) or `ctx.request_info()` (functional API) emits a request event and suspends the run until a response arrives.

Declarative workflows

Agents and workflows can also be defined in YAML for configuration management. The same declarative format underlies workflow definitions in the Foundry portal's visual builder.

5. Agent Framework in code

The examples below follow the 1.x API shapes in Microsoft's docs and package READMEs. The SDK still ships weekly, so check the official samples for exact signatures before you copy code into production.

Install

```
# Python
pip install agent-framework                    # core + common providers
pip install agent-framework-orchestrations     # Sequential, Concurrent,
Handoff, GroupChat, Magentic

# .NET
dotnet add package Microsoft.Agents.AI.Foundry
dotnet add package Microsoft.Agents.AI.Workflows
```

Your first agent

Python, backed by a Foundry project (overview):

```
import asyncio
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

agent = Agent(
    client=FoundryChatClient(

project_endpoint="https://<resource>.services.ai.azure.com/api/projects/<proj
ect>",
        model="gpt-5.4-mini",
        credential=AzureCliCredential(),
    ),
    name="HelloAgent",
```

```
    instructions="You are a friendly assistant. Keep your answers brief.",
)

async def main():
    result = await agent.run("What is the largest city in France?")
    print(result)

asyncio.run(main())
```

.NET equivalent:

```
using Azure.AI.Projects;
using Azure.Identity;
using Microsoft.Agents.AI;

AIAgent agent = new AIProjectClient(
        new
Uri("https://<resource>.services.ai.azure.com/api/projects/<project>"),
        new AzureCliCredential())
    .AsAIAgent(
        model: "gpt-5.4-mini",
        instructions: "You are a friendly assistant. Keep your answers
brief.");

Console.WriteLine(await agent.RunAsync("What is the largest city in
France?"));
```

Function tools

```
from typing import Annotated
from pydantic import Field

def get_weather(
    city: Annotated[str, Field(description="City name, e.g. Coimbatore")],
) -> str:
    """Return the current weather for a city."""
    return f"It is 29°C and sunny in {city}."

weather_agent = Agent(
    client=client,
    instructions="Answer weather questions using the tool.",
    tools=[get_weather],
)
```

```
[Description("Return the current weather for a city.")]
static string GetWeather([Description("City name")] string city) => $"29°C
and sunny in {city}.";

AIAgent weatherAgent = chatClient.AsAIAgent(
    instructions: "Answer weather questions using the tool.",
    tools: [AIFunctionFactory.Create(GetWeather)]);
```

Multi-turn sessions

```
session = agent.create_session()
await agent.run("My name is Priya.", session=session)
reply = await agent.run("What is my name?", session=session)   # remembers
"Priya"
```

```
AgentSession session = await agent.CreateSessionAsync();
await agent.RunAsync("My name is Priya.", session);
Console.WriteLine(await agent.RunAsync("What is my name?", session));
```

Middleware

```
async def audit_middleware(context, call_next):
    print(f"→ {context.agent.name} received {len(context.messages)}
message(s)")
    await call_next()
    print("← run finished")

agent = Agent(client=client, instructions="...", middleware=
[audit_middleware])
```

```
AIAgent audited = agent.AsBuilder()
    .Use(runFunc: AuditMiddleware, runStreamingFunc:
AuditStreamingMiddleware)
    .Build();
```

A graph workflow

```
from agent_framework import WorkflowBuilder, WorkflowContext, executor
from typing_extensions import Never

@executor(id="upper_case")
async def upper_case(text: str, ctx: WorkflowContext[str]) -> None:
    await ctx.send_message(text.upper())

@executor(id="reverse_text")
async def reverse_text(text: str, ctx: WorkflowContext[Never, str]) -> None:
    await ctx.yield_output(text[::-1])

workflow = WorkflowBuilder(start_executor=upper_case).add_edge(upper_case,
reverse_text).build()
result = await workflow.run("hello world")
print(result.get_outputs()[0])   # DLROW OLLEH
```

Agents drop into the same graph: `WorkflowBuilder(start_executor=writer_agent).add_edge(writer_agent, reviewer_agent).build()`.

Orchestration patterns

From the `agent-framework-orchestrations` package (PyPI):

```
from agent_framework.orchestrations import (
    SequentialBuilder, ConcurrentBuilder, HandoffBuilder, GroupChatBuilder,
MagenticBuilder,
)

sequential = SequentialBuilder(participants=[researcher, writer,
reviewer]).build()
concurrent = ConcurrentBuilder(participants=[legal, finance, tech]).build()
handoff = (HandoffBuilder()
           .participants([triage, billing, support])
           .with_start_agent(triage)
           .build())
magentic = MagenticBuilder(participants=[researcher, coder],
manager_agent=manager).build()

# Any workflow can be used as a single agent
team_agent = sequential.as_agent()
```

Harness agent

```
agent = create_harness_agent(
    client=FoundryChatClient(credential=AzureCliCredential()),
    agent_instructions="You are a research assistant. Plan your work, then
execute it.",
    tools=[],
)
response = await agent.run("Research the outlook for renewable energy
stocks.")
```

Interoperability

Protocol / integration	What it gives you
MCP	Connect local or remote MCP servers as tools; tools are discovered at run time
A2A	Call remote agents through `A2AAgent`, or expose your agent as an A2A endpoint
OpenAPI	Turn an OpenAPI spec into callable tools
AG-UI	Stream agent events to web front ends

Protocol / integration	What it gives you
GitHub Copilot SDK, Claude Agent SDK	Delegate to coding agents inside a workflow with the same identity and telemetry policies
Microsoft 365 / Copilot Studio	Surface agents in Teams and M365 Copilot; call Copilot Studio agents

Observability and dev tooling

Agent, chat and function layers emit OpenTelemetry spans, events and metrics that follow the GenAI semantic conventions; point them at Azure Monitor / Application Insights, Aspire, Jaeger or any OTLP collector. **DevUI** (preview) is a browser-based debugger that lets you chat with agents and step through workflow runs locally.

6. Foundry Agent Service: concepts

Foundry Agent Service runs agents for you on three runtime components (agents, conversations and responses) and offers two agent types: prompt agents that Foundry fully manages, and hosted agents that run your own container. The older Assistants-style model of threads, runs and `AgentID` GUIDs has been replaced; agents are now identified by name and version (runtime components).

Runtime components

Component	What it is	Use it when
Agent	A persisted, named, versioned definition: model, instructions, tools, optional safety controls	Many requests need the same behavior or tool setup
Conversation	A durable object that stores items: messages, tool calls, tool outputs	Later turns need server-side history, or a user returns across sessions or channels
Response	One unit of execution: runs a model or agent against input and appends output items	Every interaction; works with or without an agent or conversation

Responses can stream, run in background mode (poll until done) or be created with `store=false` for zero-retention scenarios, in which case you carry context forward yourself.

```
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition, WebSearchTool
from azure.identity import DefaultAzureCredential

project = AIProjectClient(endpoint=PROJECT_ENDPOINT,
credential=DefaultAzureCredential())

agent = project.agents.create_version(
    agent_name="support-agent",
    definition=PromptAgentDefinition(
        model="gpt-5-mini",
        instructions="You are a helpful support assistant.",
        tools=[WebSearchTool()],
    ),
)

openai = project.get_openai_client(agent_name="support-agent")
conversation = openai.conversations.create()
r1 = openai.responses.create(conversation=conversation.id, input="What is the
largest city in France?")
r2 = openai.responses.create(conversation=conversation.id, input="What is its
population?")
print(r2.output_text)
```

Ways to build

	Prompt agents	Hosted agents	Responses API (ephemeral)
Authoring	Portal, SDK or REST	Agent Framework, LangGraph, OpenAI Agents SDK, Anthropic Agent SDK, GitHub Copilot SDK, custom code	Your own app code

	Prompt agents	Hosted agents	Responses API (ephemeral)
Runtime code to maintain	None	Your agent logic	Your whole app
Compute	Fully managed	Per-session container sandbox, Foundry-managed	Yours
Identity	Entra agent identity	Dedicated Entra identity per agent, created at deploy	Your app's identity / OBO
Cost model	Inference + tool usage	Inference + tools + container CPU/memory	Inference + tools
Best for	Fast start, production agents without custom orchestration	Custom code, custom orchestration, multi-agent systems, webhooks, voice, AG-UI	Agent definitions that ship with your app

Source: Foundry Agent Service overview. Voice-based prompt agents (real-time speech via Voice Live) and portal-built workflow agents (YAML-backed, with branching and human-in-the-loop) also appear in parts of the documentation.

Hosted agents in detail

You package the agent as a container image (or a .zip of source that Foundry builds), push it to Azure Container Registry, and deploy. Agent Service assigns a dedicated Entra identity and endpoint, then provisions compute per session (hosted agents).

- **Isolation**: each session runs in its own VM-isolated sandbox with a persistent `$HOME` and `/files`.
- **Lifecycle**: a session is active while requests arrive, goes idle after a configurable timeout (5–60 min, default 15), and resumes on new compute with state restored. Sessions are deleted after 30 days of inactivity.
- **Sizes**: 0.5 vCPU / 1 GiB, 1 vCPU / 2 GiB, or 2 vCPU / 4 GiB per session; up to 20 GiB disk per session at 1 vCPU or more.

- **Versioning**: every version is an immutable snapshot of image, resources, environment variables and protocols. An endpoint serves one version at a time; traffic splitting is not supported.
- **Observability**: an Application Insights connection string is injected automatically, and the protocol libraries emit OpenTelemetry traces by default.
- **Languages**: Python and C#.

Protocol	Payload	Use for
Responses	OpenAI-compatible `/responses`; platform manages history and streaming	Chat assistants, RAG, background jobs; the default choice
Invocations	Arbitrary JSON via `/invocations`; you manage sessions	Webhooks (GitHub, Stripe, Jira), batch extraction, AG-UI, protocol bridges
Invocations (WebSocket)	Bidirectional stream	Real-time voice agents
Activity	Bridged automatically from Responses	Teams and Microsoft 365 channels
A2A (preview)	Agent-to-Agent protocol	Delegation between agents

Tools and toolboxes

Built-in tools include web search (Grounding with Bing), file search, code interpreter, Azure AI Search and memory; custom tools come from functions, OpenAPI specs and MCP servers; Foundry-specific tools include SharePoint, WorkIQ and Fabric IQ. A **toolbox** curates a set of tools once and exposes them behind a single managed MCP endpoint, with central authentication, governance and versioning. Agent Framework connects to it through `FoundryToolbox` (Python) or `AddFoundryToolboxes` (.NET). Tool connections authenticate by key, Entra (agent or project managed identity), OAuth On-Behalf-Of passthrough, or no auth where appropriate.

Memory

Managed memory (preview) is configured through memory stores that can keep chat summaries and user profiles across sessions, using a chat model plus an embedding model. Attach a store to an agent and stored memories become extra context at response

time.

Development lifecycle

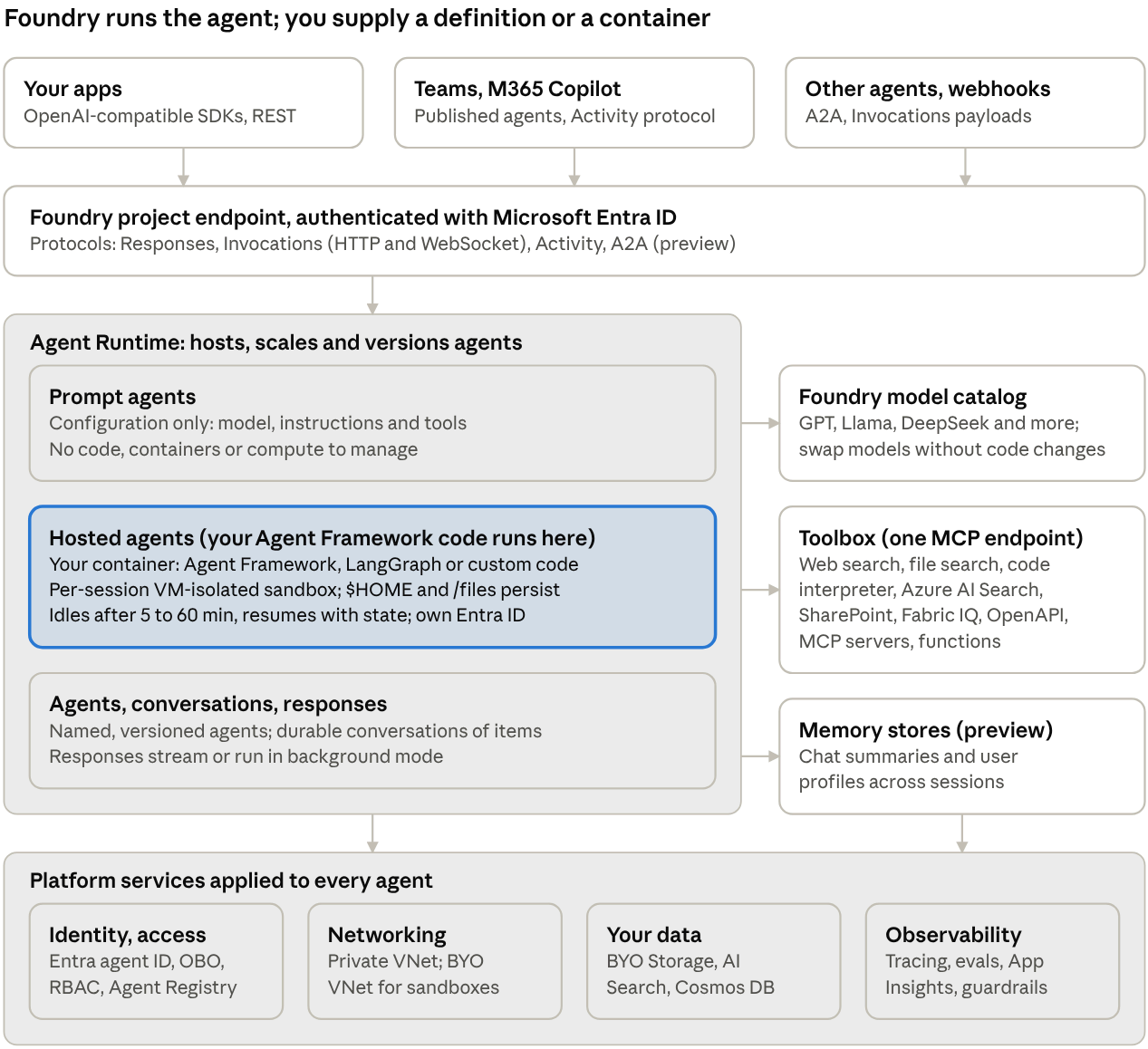
1. **Create**: define a prompt agent in the portal or SDK, or write a hosted agent.
2. **Test**: chat in the agents playground or run locally; exercise MCP tools before publishing.
3. **Trace**: inspect every model call, tool call and decision.
4. **Evaluate**: run evaluations to measure quality and catch regressions.
5. **Optimize**: the agent optimizer (preview) proposes better instructions, skills, tool descriptions and model choices.
6. **Publish**: promote to a managed resource with a stable endpoint.
7. **Monitor**: track performance and reliability on service dashboards.

Publishing and distribution

Versions are snapshotted automatically and can be rolled back. Published agents can be shared to Microsoft 365 Copilot, Teams and the Entra Agent Registry, and exposed through the OpenResponses, Activity, Invocations and A2A (preview) protocols.

7. Foundry Agent Service architecture

Every request enters through one Entra-authenticated project endpoint, runs in the Agent Runtime as a prompt agent or a hosted agent, and reaches models, tools and memory through Foundry, with identity, networking, data and observability applied by the platform.



Foundry Agent Service architecture · clients to platform services

The highlighted box is where code you write with Agent Framework lands when deployed. Microsoft publishes its own detailed hosted-agent architecture diagram and request-flow sequence (hosted agents).

Component responsibilities

Component	Responsibility	Who owns it
Project endpoint	Single entry for inference, agents and tools; Entra auth; protocol routing	Platform

Component	Responsibility	Who owns it
Agent Runtime	Hosts and scales prompt and hosted agents; manages conversations, tool calls and lifecycle	Platform
Prompt agent definition	Model, instructions, tools, guardrails	You (configuration)
Hosted agent container	Orchestration logic, custom tools, framework code	You (code)
Sandbox and session state	Per-session VM isolation, `$HOME` / `/files` persistence, scale to zero	Platform
Model catalog	Model deployments used by agents	You choose, platform serves
Toolbox	Curated, versioned tools behind one MCP endpoint with central auth	You curate, platform hosts
Platform services	Identity, RBAC, VNet, BYO resources, tracing, evaluations, guardrails	Platform, configured by you

Hosted-agent endpoints

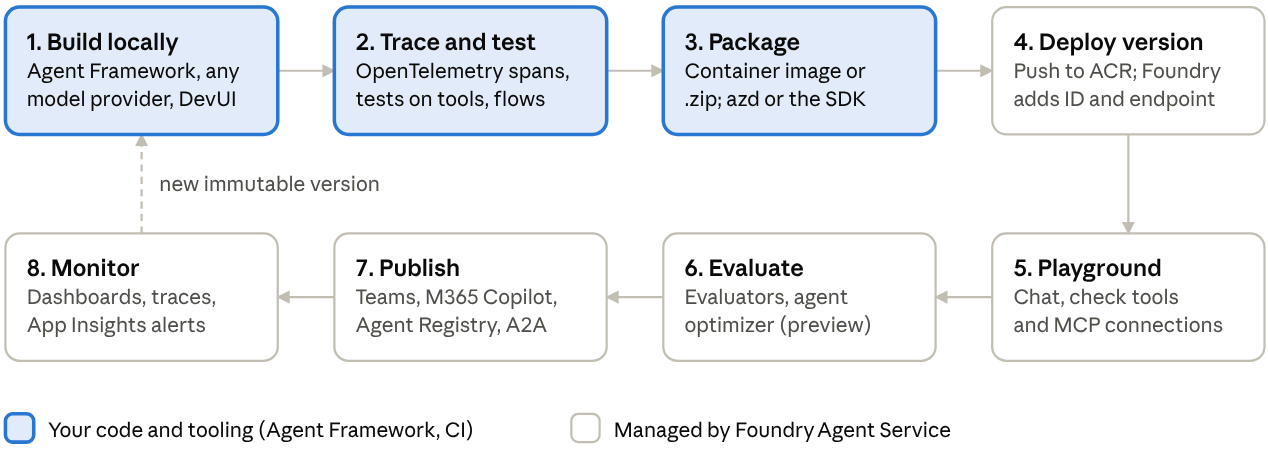
```
Responses
{project_endpoint}/agents/{name}/endpoint/protocols/openai/responses
Invocations  {project_endpoint}/agents/{name}/endpoint/protocols/invocations
WebSocket
wss://{account}.services.ai.azure.com/api/projects/{project}/agents/{name}/en
dpoint/protocols/invocations_ws?api-version=v1
A2A          {project_endpoint}/agents/{name}/endpoint/protocols/a2a
```

Endpoints are live as soon as the version deploys; publishing is only needed for distribution to channels such as Teams. Two identities are involved: the per-agent Entra identity the container uses at run time, and the project's managed identity that the platform uses for infrastructure tasks such as pulling the image.

8. How the two work together

Agent Framework and Foundry meet at three integration points: Foundry models and tools called from Agent Framework code, Foundry prompt agents used as Agent Framework agents, and Agent Framework code deployed into Foundry as a hosted agent. The same code can start with the first and move to the third without a rewrite.

One codebase moves from laptop to a governed Foundry endpoint



Build-to-production lifecycle · 8 steps

Steps 1 to 3 happen in your repo and CI; steps 4 to 8 happen in the Foundry project, and each change ships as a new immutable hosted-agent version.

Integration patterns

Pattern	Agent Framework piece	Foundry piece	Choose it when
Foundry as the model and tool backend	`FoundryChatClient` / `AIProjectClient.AsAIAgent()`	Project endpoint, Responses API, model catalog	Your agent runs in your own app or Azure service and only needs Foundry models and tools
Foundry prompt agent inside a workflow	`FoundryAgent` (direct agent type)	Prompt agent, versioned in Foundry	Business users own the prompt in the portal while developers own orchestration

Pattern	Agent Framework piece	Foundry piece	Choose it when
Agent Framework as a hosted agent	Your agent or workflow + protocol library (Responses / Invocations)	Hosted agent: sandbox, identity, scaling, endpoint	You need custom code and want Foundry to run, secure and scale it
Shared tools	`FoundryToolbox` (Python), `AddFoundryToolboxes` (.NET)	Toolbox MCP endpoint	Several agents need the same governed tool set
Shared telemetry	OpenTelemetry instrumentation	Application Insights + Foundry tracing	You want one trace across app, agent and tools

Foundry notes that calling the Responses API from your own process today and packaging the same Agent Framework code as a hosted agent later are complementary, not alternatives (Foundry overview).

Reference scenario: enterprise support assistant

1. A customer writes in Teams; the message reaches the published hosted agent over the Activity protocol, bridged to Responses.
2. Inside the container, an Agent Framework **handoff** workflow starts with a triage agent.
3. Triage hands off to a billing agent (function tools over the billing API, using the agent's Entra identity) or a tech-support agent (file search over product manuals through the toolbox).
4. A refund above a threshold triggers a **tool approval**; the workflow checkpoints and waits for a human approver.
5. The approver responds; the workflow resumes from its checkpoint and the billing agent completes the refund.
6. Every model call, tool call and handoff is traced to Application Insights; nightly evaluations score answer quality on a test set.

Deploying a hosted agent (outline)

```
# 1. Scaffold from a template and sign in
azd auth login
azd init            # pick a hosted-agent template (Agent Framework, Python
or C#)

# 2. Declare protocols, CPU/memory and environment in azure.yaml (service:
azure.ai.agent)

# 3. Build, push to Azure Container Registry and create a new agent version
azd up
```

The hosted-agent quickstart and foundry-samples repo carry the exact template names and commands.

9. Security, governance, responsible AI and pricing

Foundry supplies the platform controls (identity, network isolation, guardrails, tracing); Agent Framework supplies the in-code controls (middleware, approvals, safety patterns). You need both: Microsoft states that responsibility for responsible-AI mitigations in apps built with Agent Framework stays with you (overview).

Control layers

Concern	In Foundry Agent Service	In Agent Framework code
Identity	Dedicated Microsoft Entra agent identity; OAuth On-Behalf-Of for user-delegated calls; Entra Agent Registry	Pass `DefaultAzureCredential` / managed identity; never embed keys
Access control	Azure RBAC on who can create, invoke and manage agents; RBAC grants for the agent identity on downstream resources	Least-privilege tool sets per agent
Network	Private networking / VNet for prompt agents; BYO VNet for hosted agents with VM-isolated sandboxes per session	Restrict outbound calls in your container

Concern	In Foundry Agent Service	In Agent Framework code
Content safety	Guardrails and content filters, including cross-prompt injection (XPIA) mitigation	Input/output validation middleware; PII redaction
Tool risk	Toolbox with central auth and versioning; MCP tool allow-lists	Tool approval (human-in-the-loop) before sensitive actions; loop limits in the harness
Data	Bring your own Storage, Azure AI Search and Cosmos DB; `store=false` responses	Keep secrets out of prompts and history; use Key Vault connections
Audit	Tracing, evaluations, dashboards, Application Insights	OpenTelemetry spans for every agent, chat and tool call

Source for platform controls: Foundry Agent Service overview and hosted agents.

Main threats to design for

- **Prompt injection, direct and indirect**: hostile text in a user message, a web page, a file or a tool result tries to redirect the agent. Use guardrails, treat all tool output as untrusted data, and require approval for high-impact tools.
- **Over-privileged tools**: an agent can read or send anything its tools can. Scope each agent's identity and toolbox narrowly.
- **Data leakage to third parties**: MCP servers, non-Azure models and external agents are third-party systems with their own retention rules; review what data flows to them.
- **Runaway loops and cost**: cap iterations and tokens; the harness stops its own loop at a limit.

Pricing model

Creating and running Foundry-native prompt agents and workflows carries no extra charge; you pay for model tokens and for tools and knowledge connections (pricing page).

Cost item	How it is billed
Models	Per token, at Foundry Models rates

Cost item	How it is billed
Hosted agents	Container compute: vCPU-hours and GiB-hours across active sessions
File search	Vector storage per GB per day (first GB free)
Code interpreter	Per session
Web / custom search	Per 1,000 transactions
Memory	Short-term and long-term items stored, plus retrievals
Connectors	Separate charges and licenses (Logic Apps, Fabric, SharePoint, Bing grounding, Azure AI Search)
Agent Framework	Free (MIT open source); you pay only for what it calls

The public page renders rates per region and currency, so use the Azure pricing calculator for current figures. Because hosted-agent CPU and memory are set per session, oversizing multiplies cost by concurrency; size from Application Insights peaks.

10. Comparison with other frameworks

Agent Framework is the natural default for .NET shops and Azure-centred teams that want one supported SDK from prototype to Foundry; LangChain/LangGraph, LlamaIndex and Haystack remain strong where their specialties (ecosystem breadth, data/RAG pipelines, search pipelines) matter more than Azure integration. The comparison below is a general characterization, not a benchmark; Semantic Kernel and AutoGen are included because they are the predecessors you would migrate from.

	Microsoft Agent Framework	Semantic Kernel	AutoGen	LangChain / LangGraph	LlamaIndex	Haystack
Primary focus	Agents + graph workflows, enterprise runtime	Enterprise AI orchestration, plugins	Research-grade multi-agent conversation	General LLM app toolkit; LangGraph for stateful agent graphs	Data ingestion, indexing and RAG; agent workflows	Production search / RAG pipelines
Languages	.NET, Python, Go (preview)	.NET, Python, Java	Python, .NET	Python, JS/TS	Python, TS	Python

	Microsoft Agent Framework	Semantic Kernel	AutoGen	LangChain / LangGraph	LlamaIndex	Haystack
Multi-agent	Sequential, concurrent, handoff, group chat, Magentic, custom graphs	Agent orchestrations (now superseded)	Group chat, teams	LangGraph supervisor / swarm patterns	AgentWorkflow, multi-agent	Agent components in pipelines
Durable state / checkpoints	Superstep checkpoints, pause/resume	Limited	Limited	LangGraph checkpointers	Workflow context / state	Pipeline state
Protocols	MCP, A2A, OpenAPI, AG-UI	MCP, OpenAPI	MCP	MCP via adapters	MCP via adapters	MCP via integrations
Managed hosting	Foundry hosted agents	Via Foundry	None first-party	LangGraph Platform / LangSmith deployment	LlamaCloud (parsing, indexing)	deepset Cloud
Status	GA 1.0 (Apr 2026), LTS	Maintenance mode	Maintenance mode	Active	Active	Active

Foundry Agent Service itself is framework-agnostic: hosted agents accept Agent Framework, LangGraph, the OpenAI Agents SDK, the Anthropic Agent SDK, the GitHub Copilot SDK or custom code (Foundry overview). Choosing a framework therefore does not lock you out of Foundry hosting.

When to choose what

- **Agent Framework**: .NET or mixed .NET/Python teams; Azure, Entra and M365 integration; explicit, checkpointed multi-agent workflows; migration from Semantic Kernel or AutoGen.
- **Foundry prompt agents alone**: a single agent with built-in tools, no custom orchestration, no code to maintain.
- **LangGraph**: an existing LangChain codebase, or a team already invested in LangSmith.
- **LlamaIndex**: the hard part is ingesting, parsing and indexing messy enterprise documents.
- **Haystack**: search-centric RAG pipelines with fine-grained retrieval control.

Migrating from Semantic Kernel or AutoGen

From	Main changes
Semantic Kernel	Replace `Kernel`-coupled agents with `AIAgent` / `ChatClientAgent`; let the agent create sessions instead of wiring thread classes; register tools directly on the agent (drop `[KernelFunction]` unless needed for legacy plugins); move filters to middleware
AutoGen	Map `AssistantAgent` to `Agent`; replace team/group-chat classes with orchestration builders or `WorkflowBuilder`; use the unified message types; move to OpenTelemetry for tracing

See the official migration guide from Semantic Kernel and from AutoGen.

11. Getting started, best practices and references

Start with one prompt agent in the Foundry portal, rebuild it in Agent Framework locally, then deploy that code as a hosted agent once you need custom logic. That path exercises every layer in this document in about a day.

Getting-started checklist

- [ ] Create an Azure subscription and a Microsoft Foundry resource + project in a region that supports hosted agents
- [ ] Deploy a chat model (for example a GPT-5-mini class model) and an embedding model from the catalog
- [ ] Grant yourself the Azure AI User role on the project; sign in with `az login`
- [ ] Build a prompt agent in the portal playground with one built-in tool (web or file search)
- [ ] `pip install agent-framework` or `dotnet add package Microsoft.Agents.AI.Foundry` and run the hello-agent sample against the project endpoint
- [ ] Add a function tool, a session and a logging middleware
- [ ] Build a two-agent workflow (writer → reviewer) and inspect it in DevUI

- [ ] Turn on OpenTelemetry export to the project's Application Insights
- [ ] Package the agent as a container and deploy it as a hosted agent with the Responses protocol
- [ ] Run an evaluation, then publish to Teams / Microsoft 365 Copilot

Best practices

1. **Prefer code over agents where possible.** Use deterministic functions and workflows for well-defined steps; reserve LLM reasoning for genuinely open-ended decisions.
2. **One job per agent.** Small, specialised agents with few tools are easier to test, secure and swap than one agent with twenty tools.
3. **Write tool descriptions for the model.** Clear names, typed parameters and docstrings drive tool-selection accuracy more than prompt tweaks.
4. **Make long work resumable.** Use workflow checkpoints and hosted-agent sessions so a crash or an approval wait does not lose progress.
5. **Put humans on high-impact actions.** Require tool approval for anything that sends, pays, deletes or deploys.
6. **Trace from day one.** Turn on OpenTelemetry locally; the same spans appear in Foundry later.
7. **Evaluate continuously.** Keep a test set of real queries and run evaluations on every prompt, model or tool change.
8. **Version everything.** Pin package versions, treat agent definitions as code, and promote hosted-agent versions through dev → test → prod projects.
9. **Right-size hosted sandboxes.** Start at 1 vCPU / 2 GiB and adjust from observed peaks.
10. **Keep secrets out of prompts.** Use managed identity and Key Vault connections.

Glossary

Term	Meaning
Agent	An LLM plus instructions and tools behind a run interface
Executor	A node in a workflow graph
Superstep	One round of parallel executor execution; checkpoint boundary

Term	Meaning
Context provider	Component that injects history, memory or retrieved data before each model call
Harness	Runtime that wraps a model with tool calling, planning, compaction and approvals
Prompt agent	Configuration-only agent run fully by Foundry
Hosted agent	Your container, run by Foundry with a managed endpoint and identity
Toolbox	Curated, versioned set of tools exposed through one MCP endpoint
MCP	Model Context Protocol: standard for connecting tools and context to models
A2A	Agent-to-Agent protocol for cross-runtime agent collaboration

Sources

- Microsoft Agent Framework overview (Microsoft Learn)
- Agent concepts and Agent pipeline architecture
- Workflow concepts and Get started with workflows
- agent-framework-orchestrations on PyPI
- What is Microsoft Foundry Agent Service?
- Runtime components: agents, conversations, responses
- Hosted agents, including Microsoft's own hosted-agent architecture diagram
- Foundry Agent Service pricing
- InfoQ: Agent Framework Harness and Hosted Agents reach GA (Aug 2026)
- Visual Studio Magazine: Agent Framework 1.0 (Apr 2026)